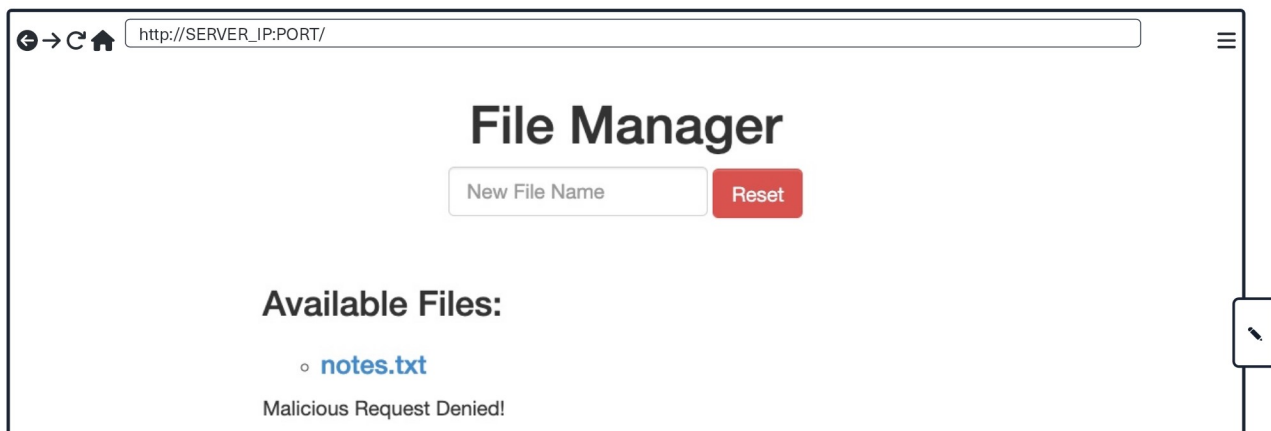


1. Bypassing Security Filters

The other and more common type of HTTP Verb Tampering vulnerability is caused by **Insecure Coding** errors made during the development of the web application, which lead to web application not covering all HTTP methods in certain functionalities. This is commonly found in security filters that detect malicious requests. For example, if a security filter was being used to detect injection vulnerabilities and only checked for injections in **POST** parameters (e.g. `$_POST['parameter']`), it may be possible to bypass it by simply changing the request method to **GET**.

2. Identify

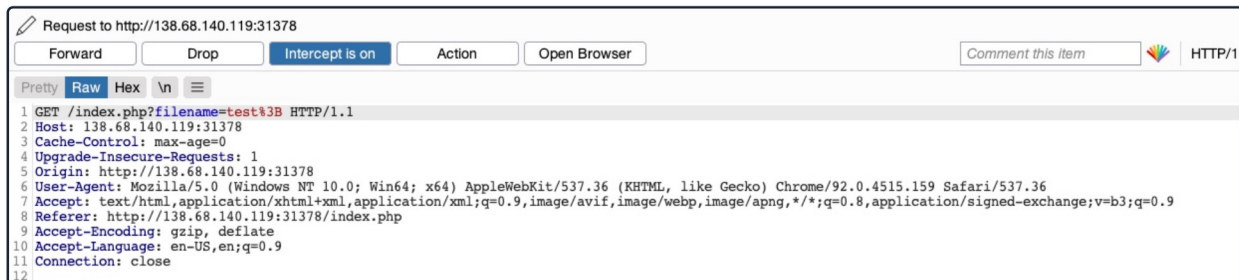
In the **File Manager** web application, if we try to create a new file name with special characters in its name (e.g. `test;`), we get the following message:



This message shows that the web application uses certain filters on the back-end to identify injection attempts and then blocks any malicious requests. No matter what we try, the web application properly blocks our requests and is secured against injection attempts. However, we may try an HTTP Verb Tampering attack to see if we can bypass the security filter altogether.

2. Exploit

To try and exploit this vulnerability, let's intercept the request in Burp Suite (Burp) and then use **Change Request Method** to change it to another method:

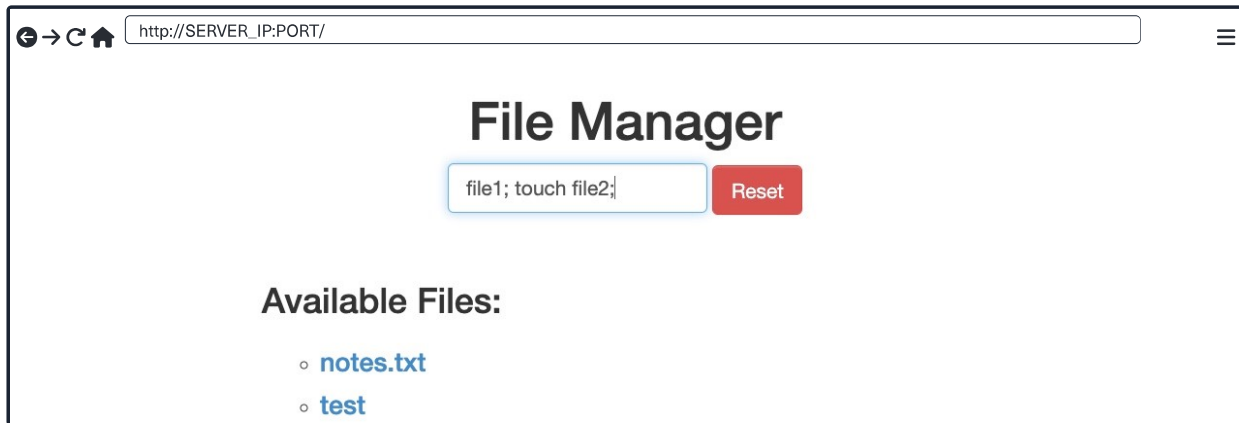


This time, we did not get the **Malicious Request Denied!** message, and our file was successfully created:





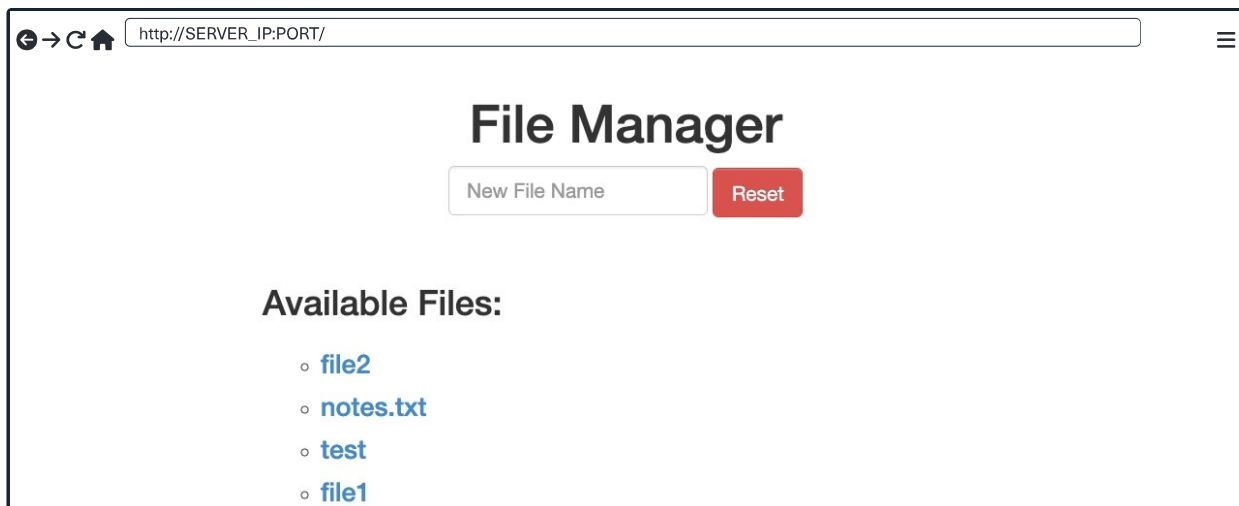
To confirm whether we bypassed the security filter, we need to attempt exploiting the vulnerability the filter is protecting: a Command Injection vulnerability, in this case. So, we can inject a command that creates two files and then check whether both files were created. To do so, we will use the following file name in our attack (`file1; touch file2;`):



Then, we can once again change the request method to a `GET` request:



Once we send our request, we see that this time both `file1` and `file2` were created:



Integrated Terminal

This shows that we successfully bypassed the filter through an HTTP Verb Tampering vulnerability and achieved command injection. Without the HTTP Verb Tampering vulnerability, the web application may have been secure against Command Injection attacks, and this vulnerability allowed us to bypass the filters in place altogether.



Connect to Pwnbox

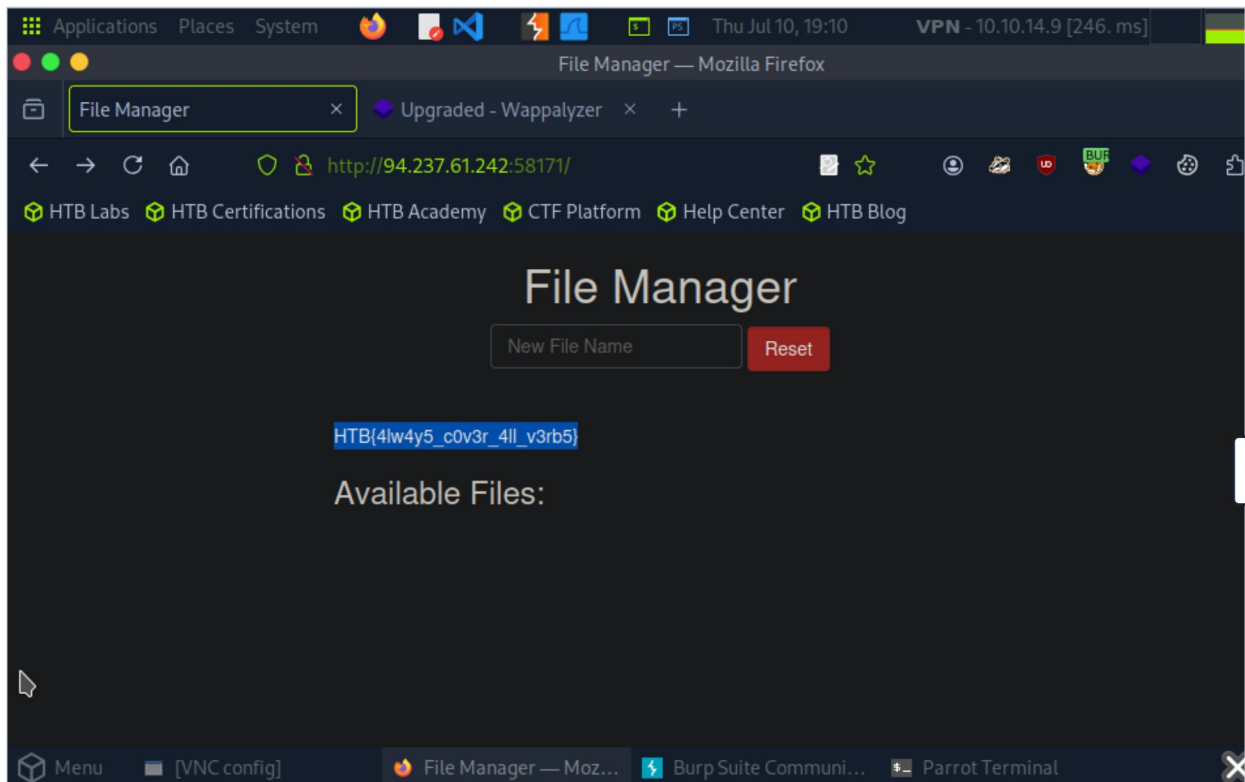
Your own web-based Parrot Linux instance to play our labs.

Pwnbox Location

AU

159ms

Terminate Pwnbox to switch location



Disconnected

Life Left: 113m



4. Questions

Answer the question(s) below to complete this Section and earn cubes!

Cheat Sheet

Target(s): 94.237.61.242:58171

Life Left: 83 minute(s)

+1 To get the flag, try to bypass the command injection filter through HTTP Verb Tampering, while using the following filename:
file; cp /flag.txt ./

Submit your answer here...

Integrated Terminal

+10 Streak pts

 Submit Hint Show Solution[← Previous](#)[Next →](#) Cheat Sheet[? Go to Questions](#)

5. Table of Contents

Introduction to Web Attacks



6. HTTP Verb Tampering

Intro to HTTP Verb Tampering

 Bypassing Basic Authentication [Bypassing Security Filters](#)

Verb Tampering Prevention

6. Insecure Direct Object References (IDOR)

Intro to IDOR

Identifying IDORs

 Mass IDOR Enumeration Bypassing Encoded References IDOR in Insecure APIs Chaining IDOR Vulnerabilities

IDOR Prevention

6. XML External Entity (XXE) Injection

Intro to XXE

 Local File Disclosure Advanced File Disclosure Blind Data Exfiltration

XXE Prevention

6. Skills Assessment

 Web Attacks - Skills Assessment

5. My Workstation

Integrated Terminal

